ERD Essentials — Entities, Attributes, Keys & Relationships

A compact, beginner■friendly reference based on the uploaded slides, with added explanations, examples, and ready■to■use SQL snippets.

1) From Data to Database
- Data are raw facts; Information is data with meaning.
- A database (usually relational) stores data in tables (rows = records, columns = attributes).
- Core relational rules:
  • Each table has a Primary Key (PK) (unique, non■NULL).
  • A cell contains exactly one value of its column's type.
  • Column meanings are consistent per column.

Why this matters: Clear keys + consistent structure make queries reliable and performance predictable.

2) Modeling Layers
- Conceptual model: Only the core entities and their relationships, abstract (M:N is allowed).
- Logical model: Adds attributes and candidate keys; you may already replace M:N by association tables.
- Physical model: Concrete for a RDBMS. Includes datatypes, PK/FK/AK, indexes, NULL vs NOT NULL, and referential actions (e.g., ON DELETE).

Tip: Design top■down (conceptual → logical → physical). Don't jump into SQL before the model is stable.

3) Naming Conventions (suggested)
- Tables: Singular, leading capital (e.g., Klant, Product, Order).
- Primary key: id (integer, surrogate, no meaning).
- Attributes: camelCase (e.g., voornaam, orderDatum).
- Foreign keys: <referencedTableSingular>Id (e.g., klantId, productId).

Rule of thumb: Choose a style and be consistent across the schema.

4) Entities & Attributes
- Entity: A thing you store info about (Klant, Product, Order…).
- Attribute: A property of an entity.
- Split composite data: e.g., voornaam + achternaam (not a single naam).
- Avoid process data: Do not store age; store geboorteDatum and compute age.

5) Keys
5.1 Primary Key (PK)
- Uniquely identifies a row; never NULL; should be stable.
- Prefer a surrogate integer id (meaningless, never edited).

5.2 Candidate & Alternate Keys
- Candidate keys: All columns (or column sets) that could serve as PK (e.g., studentenNummer, sometimes email).
- Alternate keys (AK): Non■chosen candidate keys; enforce with UNIQUE.

5.3 Why surrogate keys?
- Natural keys may change (typos, business updates).
- Integers are compact and fast for joins; reduce cascading updates.

6) Relationships & Foreign Keys (FK)
- Foreign Key links a child row to its parent row's PK.
- Cardinality:
  • 1:1: rare—often merge into one table unless there's a good reason.
  • 1:N: most common. The FK is on the N (*) side.
  • M:N: never directly; use an association table with two FKs.
- Optionality (Aard):
  • 1 (mandatory) → NOT NULL FK.
  • 0 (optional) → FK allows NULL.
- Referential actions: ON DELETE RESTRICT | CASCADE | SET NULL.

Golden rule: Place the FK on the * side of a 1:N relation.

7) Worked Example — Shop Schema
7.1 Entities
- Klant(id, voornaam, achternaam, email)
- Product(id, naam, prijs)
- Order(id, orderDatum, klantId)
- OrderProduct(orderId, productId, aantal) ← association table (M:N)

7.2 SQL (portable)

```
CREATE TABLE Klant (
  id        INT PRIMARY KEY,
  voornaam   VARCHAR(80)  NOT NULL,
  achternaam VARCHAR(120) NOT NULL,
  email      VARCHAR(255) NOT NULL UNIQUE  -- AK
);

CREATE TABLE Product (
  id    INT PRIMARY KEY,
  naam  VARCHAR(120) NOT NULL,
  prijs DECIMAL(10,2) NOT NULL
);

CREATE TABLE "Order" (
  id        INT PRIMARY KEY,
  orderDatum DATE NOT NULL,
  klantId    INT  NOT NULL,
   CONSTRAINT fk_order_klant
     FOREIGN KEY (klantId) REFERENCES Klant(id)
    ON DELETE RESTRICT
);

CREATE TABLE OrderProduct (
  orderId   INT NOT NULL,
  productId INT NOT NULL,
  aantal    INT NOT NULL CHECK (aantal > 0),
   PRIMARY KEY (orderId, productId),
   CONSTRAINT fk_op_order
     FOREIGN KEY (orderId) REFERENCES "Order"(id) ON DELETE CASCADE,
   CONSTRAINT fk_op_product
     FOREIGN KEY (productId) REFERENCES Product(id) ON DELETE RESTRICT
);
```

7.3 Queries you'll actually run

```
-- 1) Alle orders met klantnaam
SELECT o.id, o.orderDatum, k.voornaam, k.achternaam
FROM "Order" o
JOIN Klant k ON k.id = o.klantId
ORDER BY o.orderDatum DESC;

-- 2) Orderregels met productnaam en subtotaal
SELECT op.orderId, p.naam, op.aantal, (op.aantal * p.prijs) AS subtotaal
FROM OrderProduct op
JOIN Product p ON p.id = op.productId
ORDER BY op.orderId;

-- 3) Totaal per order
SELECT op.orderId,
     SUM(op.aantal * p.prijs) AS totaal
FROM OrderProduct op
JOIN Product p ON p.id = p.id
GROUP BY op.orderId
ORDER BY op.orderId;
```

Check: Begrijp je waarom klantId NOT NULL is in Order? (hint: elke order hoort bij precies één klant.)

8) Optional Relationship — Book ↔ Series (0..1)

```
CREATE TABLE Series (
  id   INT PRIMARY KEY,
  name VARCHAR(200) NOT NULL
);

CREATE TABLE Book (
  id       INT PRIMARY KEY,
  title    VARCHAR(200) NOT NULL,
  seriesId INT NULL,            -- optioneel (aard = 0)
    CONSTRAINT fk_book_series
      FOREIGN KEY (seriesId) REFERENCES Series(id)
     ON DELETE SET NULL
);
```

9) Classic M:N — Student ↔ Vak

```
CREATE TABLE Student (
  id            INT PRIMARY KEY,
  voornaam        VARCHAR(80)  NOT NULL,
  achternaam      VARCHAR(120) NOT NULL,
  studentenNummer  VARCHAR(20)  NOT NULL UNIQUE -- AK
);

CREATE TABLE Vak (
  id   INT PRIMARY KEY,
  naam VARCHAR(120) NOT NULL
);

CREATE TABLE StudentVak (
  studentId INT NOT NULL,
  vakId     INT NOT NULL,
  jaar      SMALLINT NOT NULL,
    PRIMARY KEY (studentId, vakId),
    FOREIGN KEY (studentId) REFERENCES Student(id) ON DELETE CASCADE,
    FOREIGN KEY (vakId)     REFERENCES Vak(id)     ON DELETE RESTRICT
);
```

10) Checklists
ERD Checklist
[ ] Elke tabel heeft PK (id INT, betekenisloos).
[ ] Bepaal AK's (natuurlijke uniciteit) en zet UNIQUE.
[ ] FK staat aan de *■kant (1:N) en krijgt een duidelijke naam.
[ ] M:N altijd via associatietabel met samengestelde PK.
[ ] Optionaliteit vertaald naar NULL vs NOT NULL op FK.
[ ] Vermijd procesdata en splits samengestelde velden.
[ ] Consistente naamgeving en datatypes.

Referential Actions — keuzehulp
- RESTRICT: je wil geen weeskinderen; verwijderen blokkeren als er kinderen zijn.
- CASCADE: je wil dat afhankelijke rijen automatisch meeverwijderen.
- SET NULL: relatie is optioneel; bij verwijderen wordt de FK NULL.

11) Common Pitfalls
- Betekenisvolle PK's (RRN, serienummer) → gevoelig voor wijziging → kies id.
- Duplicatie (berekenbaar/overerfbaar) → normaliseer.
- FK aan de verkeerde kant → volg de *■kant■regel.
- Onnodige directe relaties als er al een impliciet pad is.

12) Mini■Practice
1) Modelleer Klas (1) — (*) Student. Waar komt de FK en is die NULL of NOT NULL als elke student verplicht in precies één klas zit?

Appendix — Quick Notation (ASCII)
Klant (1) ■■■< (N) Order
  id PK id PK
  klantId FK → Klant.id

Student (N) >■■■< (N) Vak via StudentVak
StudentVak PK(studentId, vakId)